Orca-RL in Stonefish: obstacle-aware RL navigation over ROS 2

Vijay Srinivas P (CB.SC.U4AIE23263) · Joel Sahay D (CB.SC.U4AIE23234) · D.K. Preetham Reddy (CB.SC.U4AIE23222) · A Prithvi (CB.SC.U4AIE23201) | Guide: Dr. D. Palmani

Batch and Group Number: C-7

Date of the Presentation:

Outline

Part A · Why and how: RL

- Real-world situations and why this method
- The RL objective and reward
- How the objective is optimised
- Diagnosis of the shipped policy
- Training and offline results

Part B · The simulation

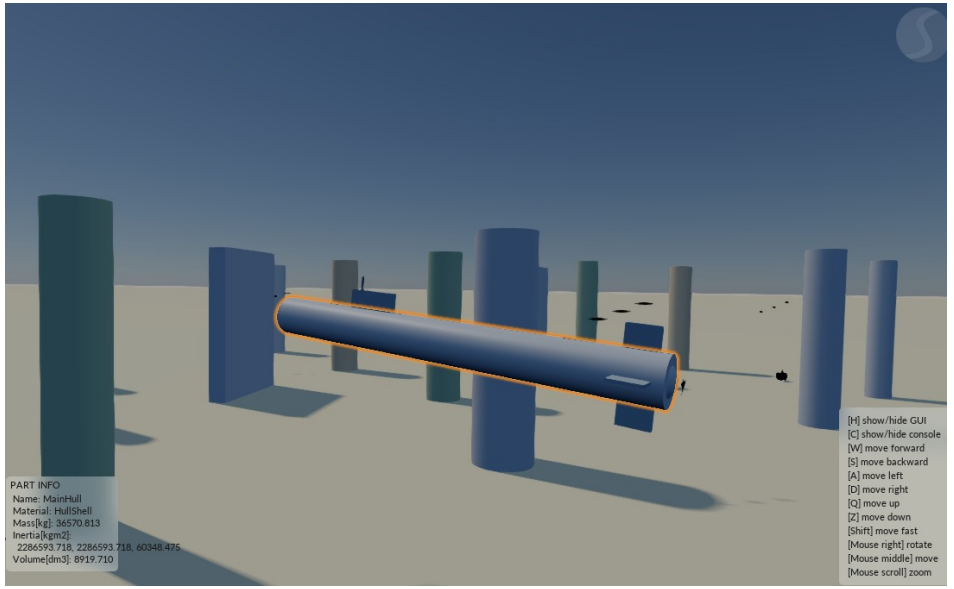
- Stonefish and the Orca model
- The base world with obstacles
- Fixing the vehicle model
- System identification
- From action to thruster force

Part C · Connecting them

- Architecture over ROS 2
- From Gazebo to Stonefish
- Data exchanged, frames, timing
- Closed-loop results in 3 worlds
- Summary, limits, next phase

Introduction and Motivation

- An extra-large uncrewed submarine such as the **Boeing Orca** has to survey long distances **without GPS**, with a position estimate that degrades underwater.
- The mid-review delivered a reinforcement-learning stack (physics, sensors, particle filter, safety shield, trained policy) and, separately, a **Stonefish** simulator with survey scenarios.
- The two tracks were **not connected**: the bridge between them assumed a different simulator (Gazebo) and the Stonefish scenes used a hand-written reactive controller.
- This phase connects them: the RL policy now **drives the Stonefish Orca over ROS 2** and avoids rocks, fish and other submarines **using only its own sonar**.



Stonefish: the Orca among rock pinnacles in the obstacle world

Real-World Situations

Charts are incomplete

- 2005: a US Navy submarine struck an undersea mountain near Guam that was missing from its charts
- 2021: another struck an uncharted seamount in the South China Sea
- A pre-planned route is not enough: the vehicle must sense and react

No GPS under water

- Below the surface only inertial and Doppler (DVL) dead reckoning is available
- The error grows with distance; a long-range vehicle cannot keep surfacing
- Decisions must be made on an uncertain estimate

Long survey missions

- Seabed mapping, pipeline and cable routes, wind-farm sites, search operations
- 2014: deep-sea vehicles searched the seabed for weeks for flight MH370
- Lawnmower lanes, hours of autonomy, limited battery

Shared, moving water

- Fish schools, marine mammals, other survey vehicles, ship traffic
- Obstacles appear without warning and move
- Losing a vehicle is costly: Autosub2 was lost under an Antarctic ice shelf in 2005

Why This Method Is Proposed

Real-world situation	Design response in this work
No GPS under water: position from dead reckoning drifts	Navigate on the estimate, not the truth: DVL + IMU + particle filter in the loop
Charts are incomplete: subs have hit uncharted seamounts (2005, 2021)	Sense and avoid on board: forward sonar sectors + flank memory in the observation
Shared, moving waters: fish, other AUVs, traffic crossing the route	Learn reactive trade-offs end to end instead of hand-tuning a planner
Hard limits must never be broken: collision, depth, seabed, battery	Safety shield with hard rules around the learned policy; overrides are penalised
Long missions on a battery budget, no operator in the loop	Energy and idling in the reward; small network that runs on board

Why reinforcement learning

- A hand-tuned planner trades goals against clearance poorly: it collides in 28% of test missions, the learned policy in 13%.

Why a safety shield

- Learned behaviour is not a guarantee; hard rules wrap it.

Why simulation first

- Collisions cannot be tested on a 50-tonne vehicle; Stonefish gives real physics and sensors.

Each situation on the left maps to one design element of this work.

Literature Review

Sl No	Title and year	Venue	Methodology	Key findings	Limitations
1	Evolution Strategies as a Scalable Alternative to RL (2017)	arXiv / OpenAI	Antithetic Gaussian perturbations, rank-shaped fitness	Matches policy-gradient RL on control; trivially parallel	Sample-hungry; slow from scratch on sparse tasks
2	Proximal Policy Optimization (2017)	arXiv / OpenAI	Clipped surrogate policy gradient	Stable on-policy RL baseline	Needs automatic differentiation and a deep-learning toolchain
3	Handbook of Marine Craft Hydrodynamics (2011)	Wiley (Fossen)	Rigid-body + hydrodynamic modelling	Standard UUV dynamics used in our fast simulator	Parameters for Orca not public
4	Stonefish: open-source marine robotics simulator (2019)	IEEE/OES OCEANS	Bullet physics + hydrodynamics + sonar/DVL models	Realistic sensors, ROS interface	Scripted bodies never yield; no run-time reset
5	UUV Simulator (2016)	IEEE/MTS OCEANS	Gazebo plugins for UUVs	Common ROS UUV stack	Gazebo Classic; the old bridge assumed it
6	DAgger: imitation learning with no-regret online learning (2011)	AISTATS	Iterative relabelling of learner-visited states	Fixes covariate shift of behaviour cloning	Needs an expert to query

Objectives

1. Make the Stonefish application ROS 2-aware and drive the real Orca thrusters
2. Rewrite the Python bridge for Stonefish: real sensors in, physical setpoints out
3. Keep the RL methodology: localisation, health monitoring, safety shield, network + ES
4. Give the policy goal and sonar inputs and train it to avoid obstacles
5. Add obstacles, fish schools and a traffic submarine to the base world
6. Verify step by step, and report honestly where it fails

Part A · RL Formulation

State (observation)

- Estimated position, depth, heading
- Uncertainty, battery, thruster wear, navigation residual, depth margin
- Terrain richness, risk
- New: goal bearing and distance, sonar sectors, flank clearances, depth error

Action

- $a \in [-1, 1]^3$ from a 2 x 64 tanh network
- speed, yaw rate, depth rate
- scaled to what the vehicle can do
- then the safety shield, then ROS 2

Reward

- + progress toward the waypoint
- - uncertainty, - risk, - energy
- - shield overrides, - idling
- + waypoint bonus
- - collision (episode ends)

The RL Objective: What Is Optimised

$$\theta^* = \arg\max_{\theta} J(\theta), \qquad J(\theta) = \mathbb{E}_{\text{world} \sim \mathcal{D}} \left[\sum_{t=0}^{T} r_t \right], \qquad a_t = \pi_\theta(o_t)$$

$$\pi_\theta(o) = \tanh(W_3 \tanh(W_2 \tanh(W_1 o + b_1) + b_2) + b_3), \qquad \theta = \{W_k, b_k\} \in \mathbb{R}^{5891}$$

What is optimised

- θ: the 5,891 weights and biases of the policy network
- J(θ): the expected total reward of one mission
- Expectation over random worlds: start, obstacles, fish, currents, waypoints

Episode

- 1 s per step, up to 420 steps (7 min), 3 waypoints
- Ends at the last waypoint, on collision or on time-out
- Undiscounted total reward: missions are finite

The Reward Function, Term by Term

$$r_t = w_p\,\Delta d_t \;-\; w_u\,\frac{\sigma_t^2}{100} \;-\; w_r\,\rho_t \;-\; w_e\,\frac{E_t}{100} \;-\; w_h\,\mathbf{1}[\text{shield}] \;-\; w_i\,\mathbf{1}[u_t < 0.2] \;+\; w_g\,\mathbf{1}[\text{goal}] \;-\; w_c\,\mathbf{1}[\text{collision}]$$

Term	Weight	What it measures	Real-world meaning
Progress Δd	1.0 per m	drop in estimated distance to the waypoint	get the survey done
Waypoint bonus	200	waypoint reached (within 10 m)	mission completion
Uncertainty σ²/100	0.0025	spread of the particle filter (m^2)	stay localisable without GPS
Risk ρ	0.4	sonar proximity and low altitude, 0 to 1	keep a safe margin
Energy E/100	0.02	watt-hours used this step	battery endurance
Shield	1.5	the safety shield had to override	do not lean on the safety net
Idle	0.5	speed below 0.2 m/s	do not stall to avoid penalties
Collision	300	hull touched an obstacle; episode ends	losing the vehicle

From Action to a Safe Command

$$v = \frac{a_0 + 1}{2} v_{\max}, \qquad \dot{\psi} = a_1 \dot{\psi}_{\max}, \qquad \dot{z} = a_2 \dot{z}_{\max}, \qquad (v, \dot{\psi}, \dot{z}) \leftarrow \text{Shield}(v, \dot{\psi}, \dot{z} \mid \text{sonar}, \text{altitude}, \text{battery}, \sigma^2)$$

Limits from the real vehicle model

- Top speed 1.1 m/s (40 kN main thruster)
- Yaw rate up to 0.045 rad/s, less at higher speed
- Depth rate up to 0.3 m/s
- Identified on the Stonefish Orca, not assumed

Shield applied after the policy

- Original rules: depth rating, battery reserve, localisation
- New rules: obstacle stop / slow, seabed floor, depth band
- Uses sensor-derived quantities only
- Each override costs reward (the shield term)

How the Objective Is Optimised

1 Warm start: imitate a classical planner (behaviour cloning + one DAgger round)

$$\theta_0 = \arg\min_{\theta} \frac{1}{N} \sum_{j=1}^{N} \left\| \pi_\theta(o_j) - a^{\text{planner}}(o_j) \right\|^2$$

111k planner samples + 25k relabelled states; Adam, mean-squared error

2 Optimise J(θ) directly with Evolution Strategies

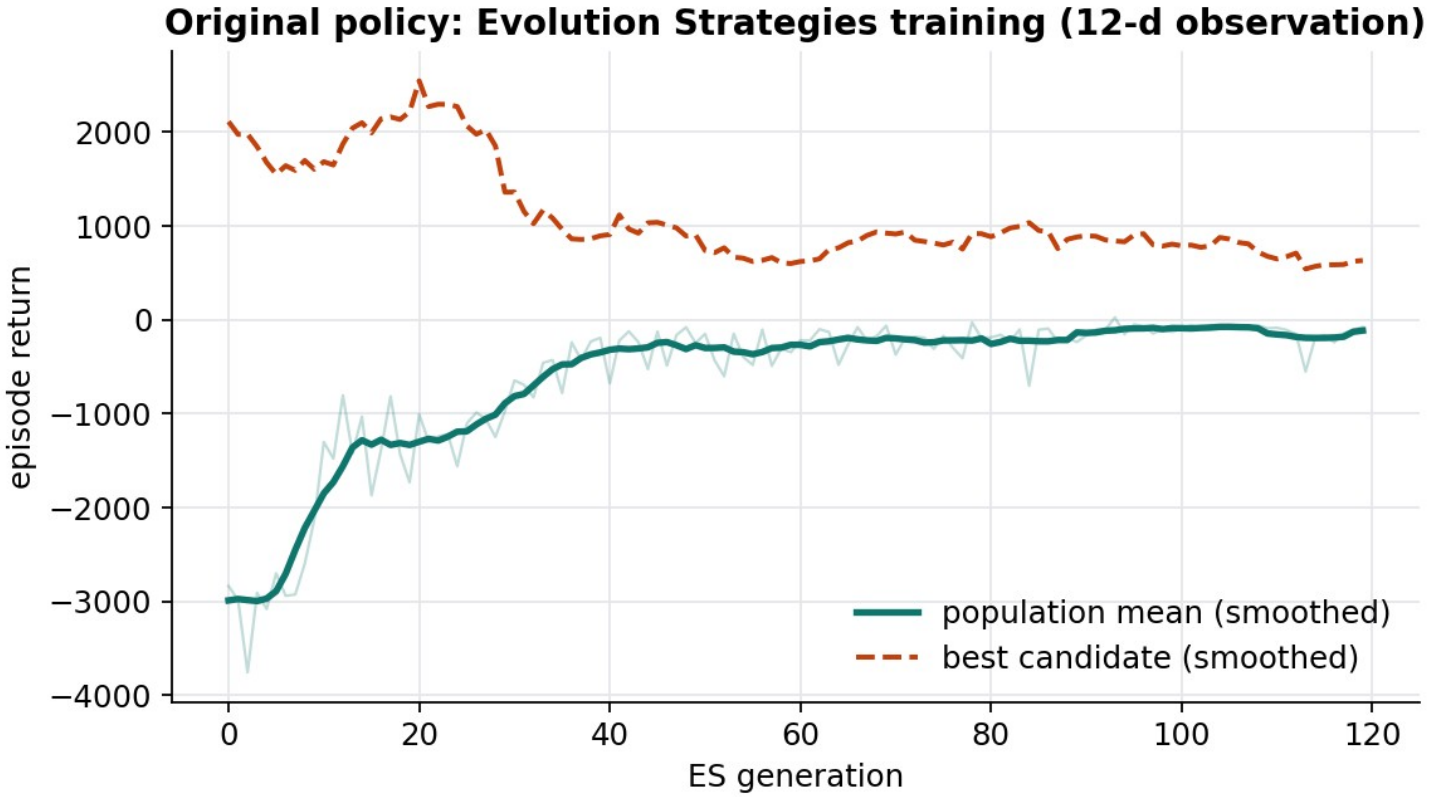
$$\hat{g} = \frac{1}{n\sigma} \sum_{i=1}^{n} \mathcal{R}(J(\theta + \sigma\varepsilon_i))\, \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, I), \qquad \theta \leftarrow \theta + \alpha\, \hat{g}$$

- No gradient through the simulator is needed; works with the abrupt collision penalty.
- Every candidate is an independent mission, so 14 cores evaluate in parallel.

Settings

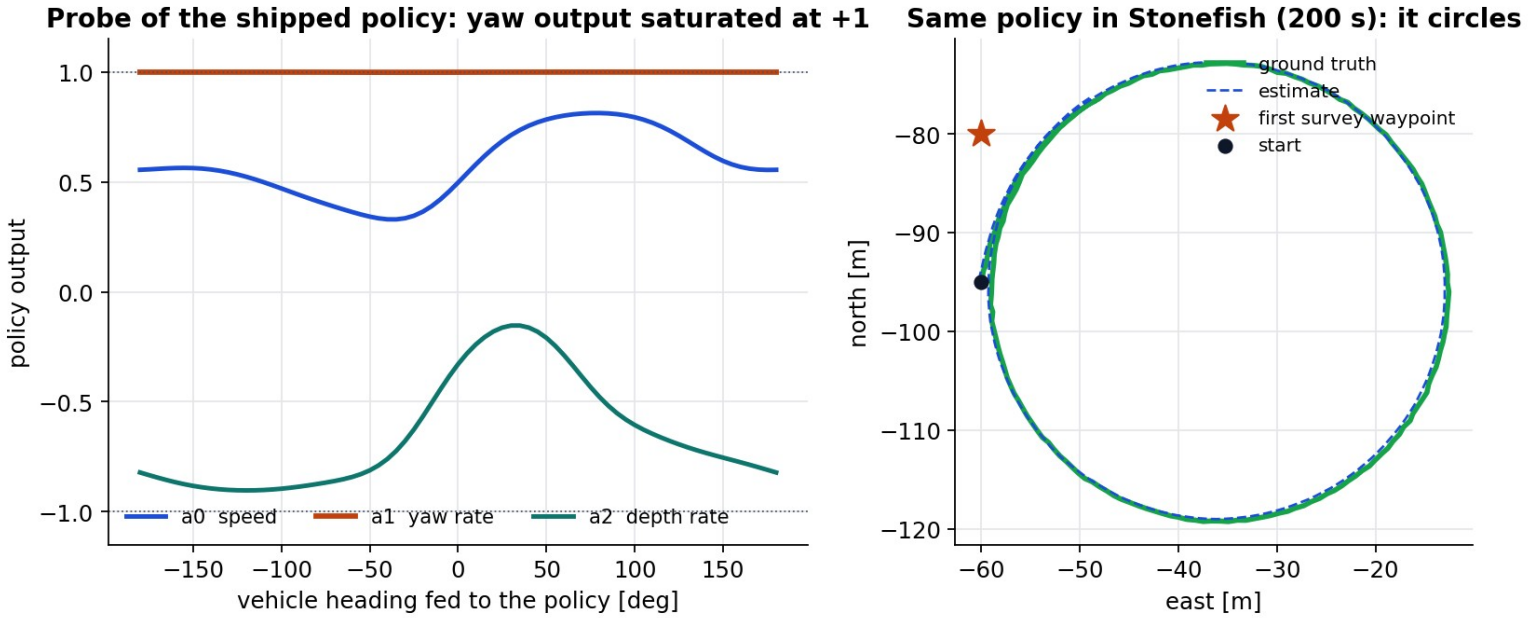
- n = 48 (24 mirrored pairs)
- σ = 0.03, α = 0.01, 200 generations
- 4 shared episodes per candidate
- rank shaping R(·) in [-0.5, 0.5]

The Shipped Policy: Training



- Trained with **Evolution Strategies** for 120 generations on the original 12-input environment.
- Mean return rises from -3000 to about -100: the population learns to **avoid penalties**, not to reach goals.
- We kept this policy unchanged; it can still be selected at run time.

Diagnosis: Why It Cannot Navigate



0 / 5

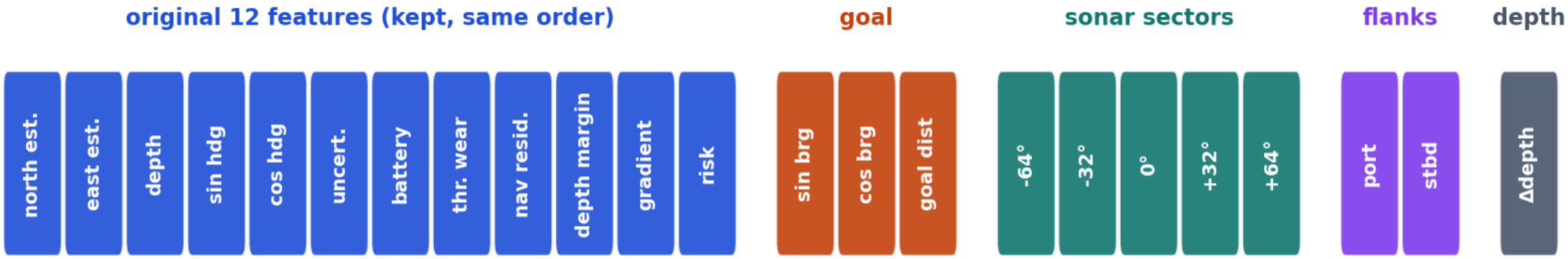
goals reached in its own evaluation

+1.0

yaw output, for every heading

- The observation contains **no goal and no obstacle information**, so no policy trained on it can steer to a waypoint or around a rock.
- In Stonefish it does exactly what the probe predicts: a tight circle near the start (1 of 10 waypoints in 200 s).

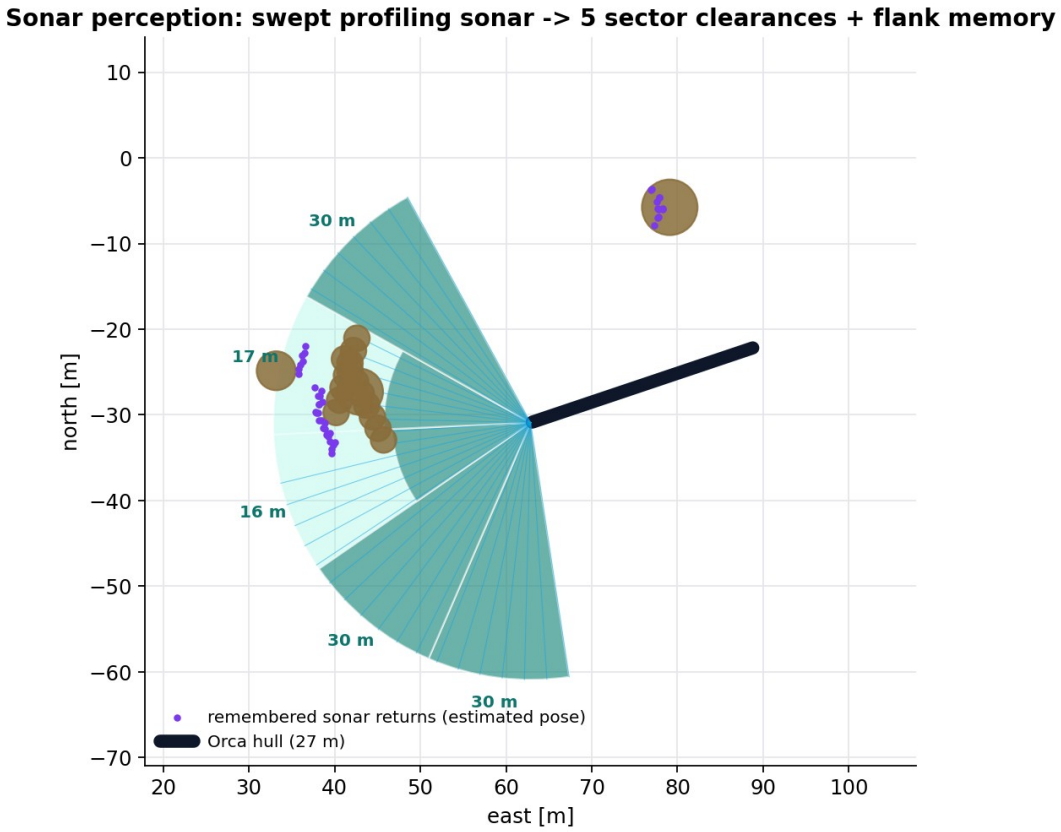
A Sonar-Aware Observation



Obstacle-aware observation: 23 inputs to the same 2 x 64 tanh network (the original policy used the first 12)

- The **first 12 inputs are unchanged**, same order and formulas.
- +3 goal inputs from the next waypoint, using the **estimated** position.
- +5 profiling-sonar sector clearances (port to starboard) measured from the bow.
- +2 hull-flank clearances from a short-term **sonar memory**.
- +1 depth error relative to the mission depth.
- Same network type: 23 -> 64 -> 64 -> 3, tanh, about 5.9k parameters.

Stonefish-Scale Training Environment



Fast training environment (offline only)

- Vehicle limits **identified on the Stonefish Orca**: 1.1 m/s, speed-dependent yaw rate, time constants.
- 27 m x 1.3 m **hull** for collisions; rocks, ridge walls, fish and subs that move.
- The **same swept sonar** as the model: 64 steps over 160° at 10 Hz, in 5° bins as the simulator publishes.
- The sonar memory keeps rocks that left the beam; a later beam that sees through a point clears it.
- Navigation = DVL dead reckoning into the existing particle filter, exactly as on the vehicle.

Safety Shield: Original Rules + Obstacle Rules

Rule	Trigger (sensor-derived only)	Override
Depth rating (original)	depth ≥ 98 % of rating	climb
Battery reserve (original)	battery < 8 %	limp-home speed, damped yaw
Localisation (original)	covariance trace > 5000 m^2	clamp yaw rate
Obstacle stop	something < 7 m dead ahead	crawl 0.15 m/s, turn hard to the open side
Obstacle slow	something < 12 m in the front sectors	cap speed at 0.45 m/s
Depth band	|depth - mission depth| > 3 m	push back into the band

Every override costs the policy reward, so it learns to avoid needing the shield; the same shield runs in training and on the vehicle.

Training Pipeline: Behaviour Cloning + ES



same MLP (23-64-64-3, tanh), same ES trainer, same reward terms + collision penalty, safety shield in the loop, 14 worker processes (65 min)

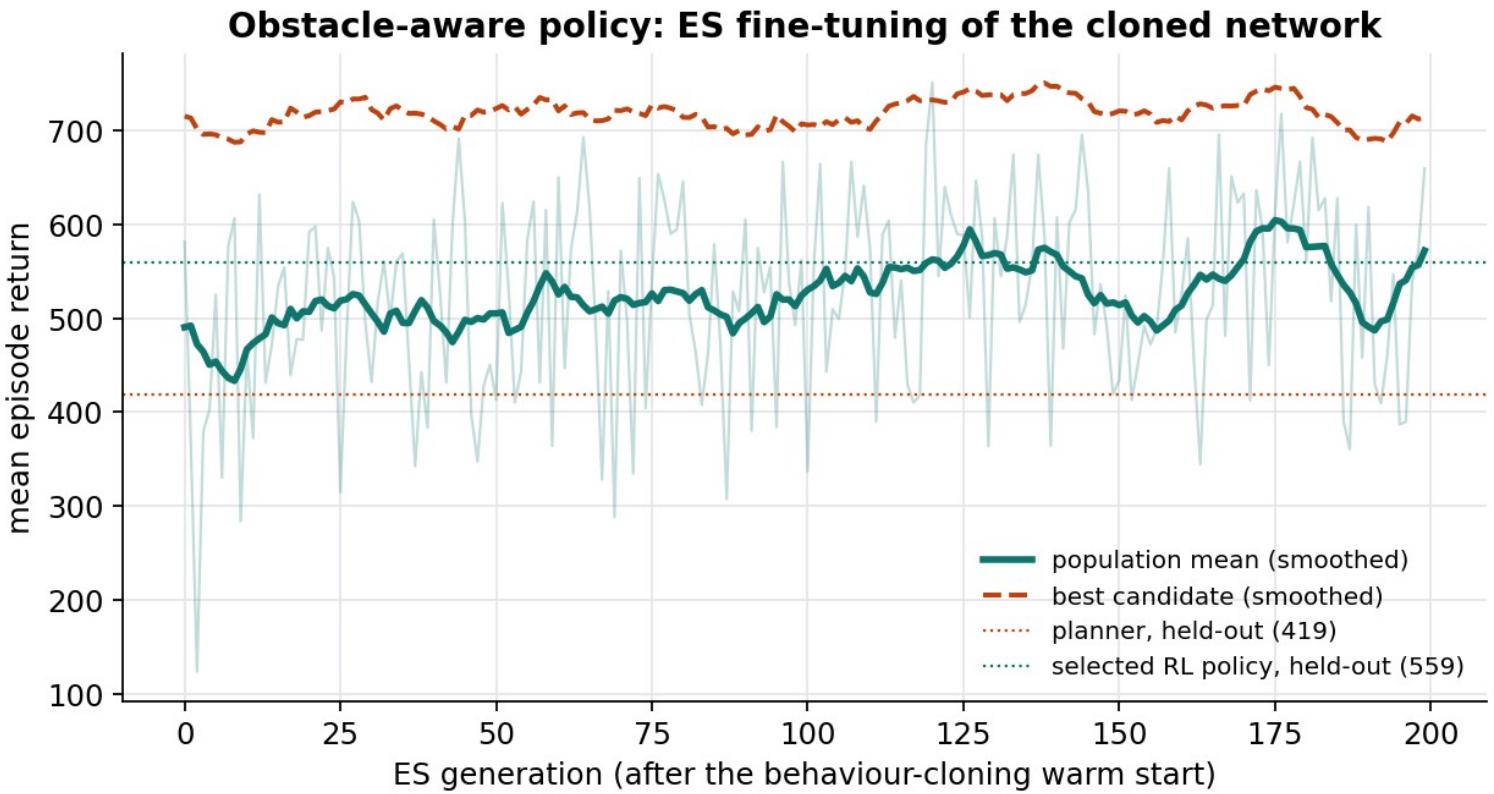
Why a warm start

- ES from a random network reached only 8 % of goals after 50 generations
- Cloning a classical planner gives a sensible starting behaviour

What ES then optimises

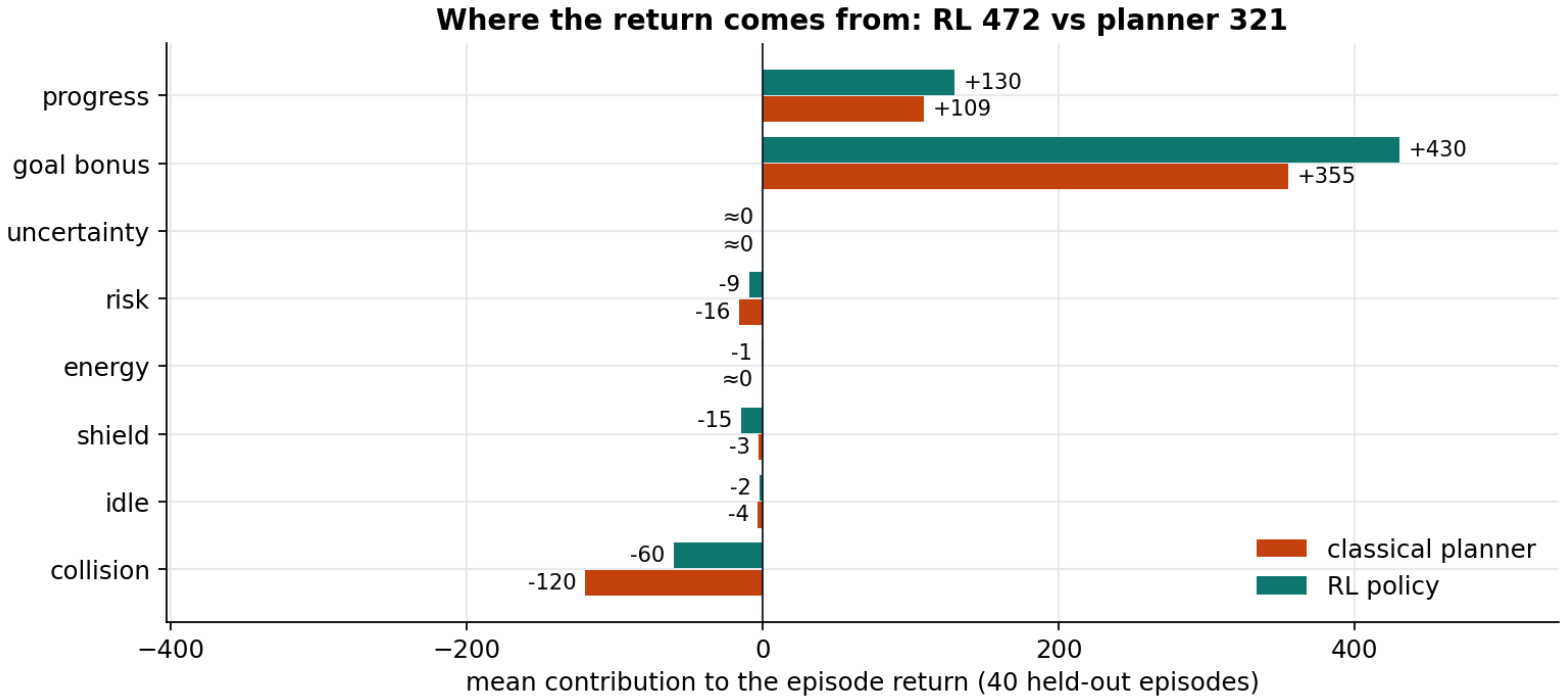
- The real reward, collisions included, with $\sigma = 0.03$
- Shared evaluation episodes per generation, 14 workers in parallel

ES Fine-Tuning of the Cloned Policy



- 200 generations x 48 candidates x 4 episodes (65 min on 14 cores).
- Generation-to-generation noise comes from new episodes each generation.
- Model selection on 100 **validation** missions; a later 80-generation fine-tune was worse (16 % vs 8 % collisions) and was **rejected**.

Where the Reward Goes

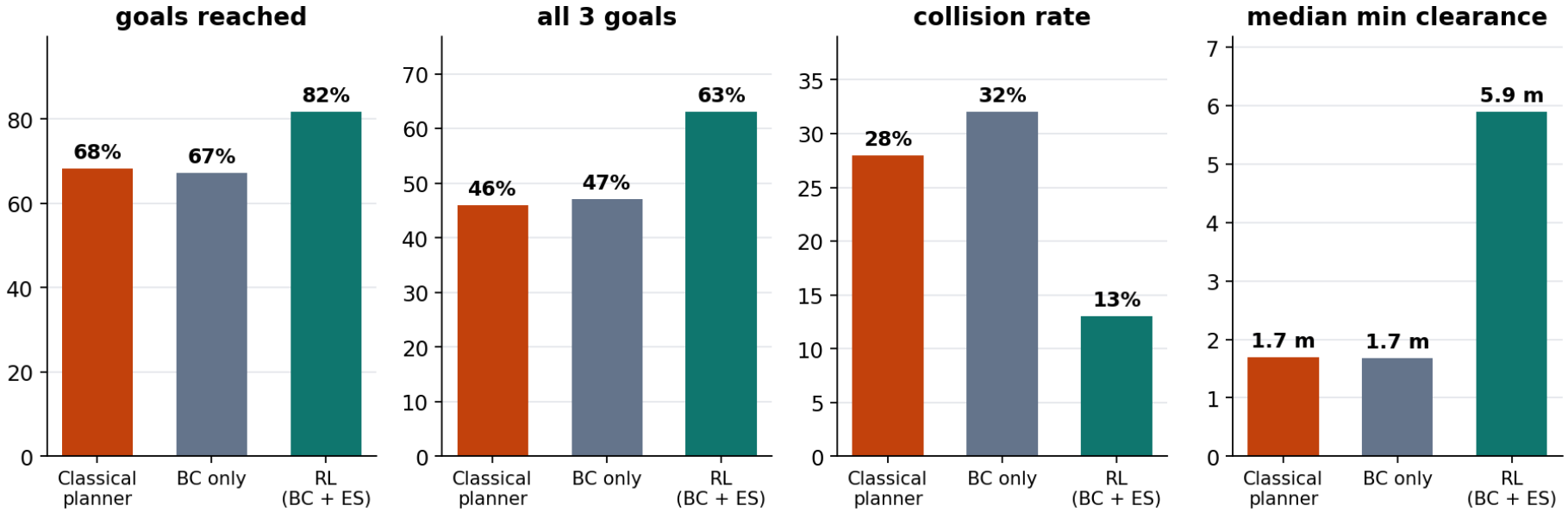


Mean contribution of each reward term over 40 held-out missions (energy approximated from speed).

- The RL policy earns more **waypoint bonus** (430 vs 355) and more **progress** (130 vs 109).
- It **halves the collision penalty** (-60 vs -120).
- It pays a little more for shield overrides: it accepts brief interventions to keep moving.
- Uncertainty and energy terms are small at this scale (flat seabeds, 7-minute missions).

Offline Results: 100 Unseen Missions

Held-out test: 100 unseen obstacle episodes (seeds 7000+)



82 %

goals reached (planner 68 %)

13 %

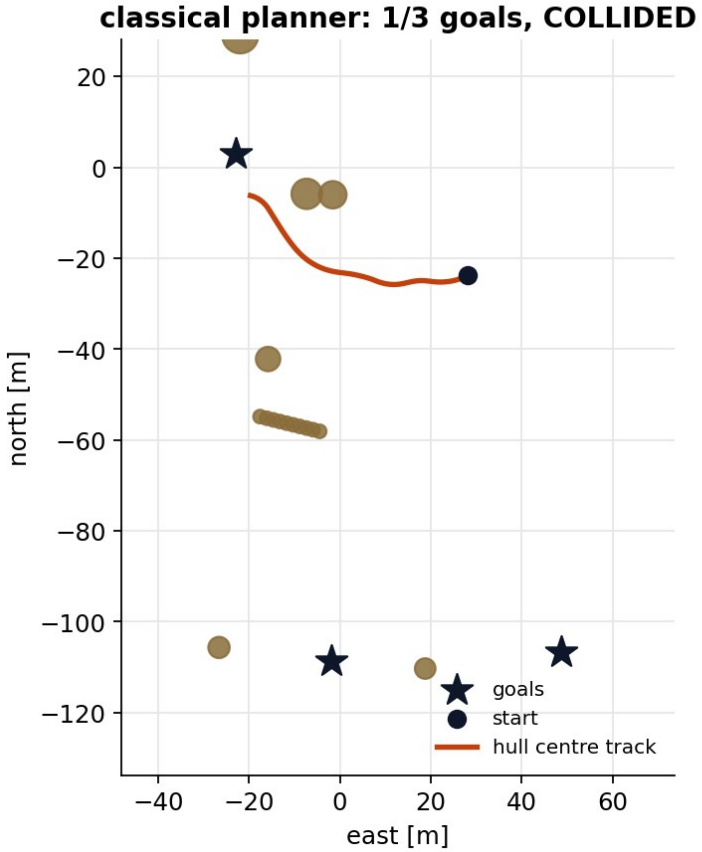
collision rate (planner 28 %)

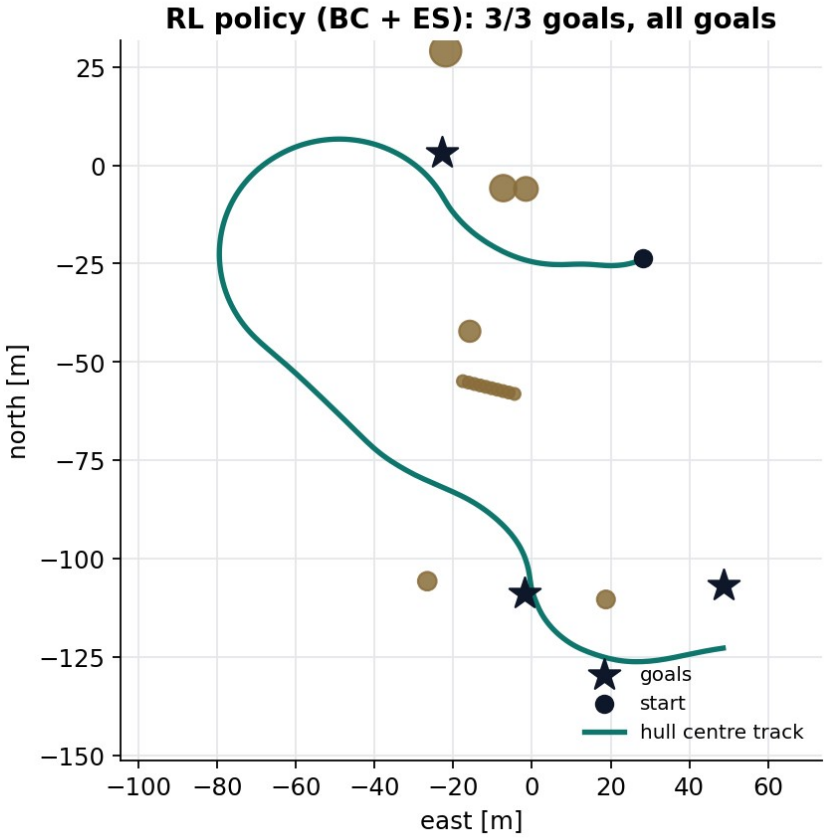
5.9 m

median closest pass (planner 1.7 m)

Same Mission, Two Controllers

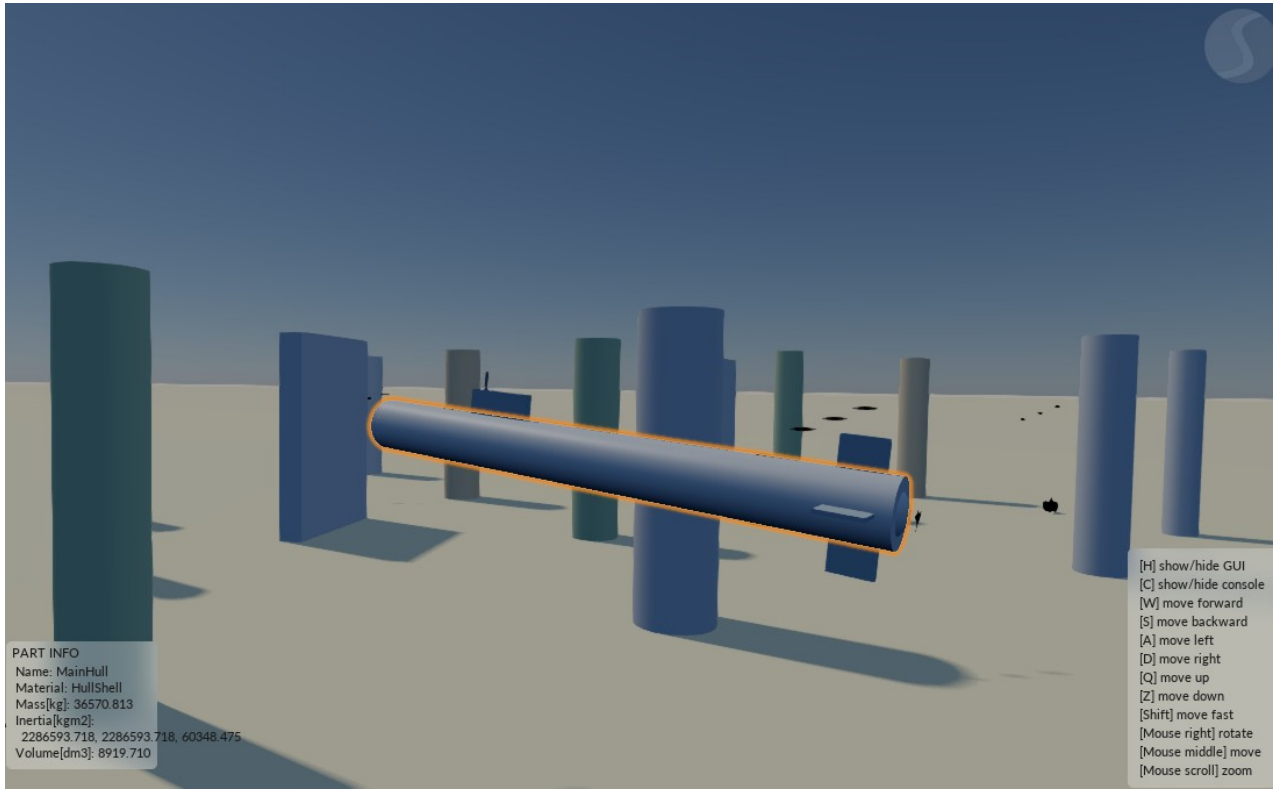
Training environment, held-out seed 7010 (rocks brown, moving fish/subs blue)





- Classical planner: attraction to the goal, repulsion from close sectors.
- It cuts the corner and hits the rocks beside the first goal.
- The RL policy slows down, swings wide around the cluster and completes all three goals.
- Both read the **same observation**; only the decision rule differs.

Part B · Stonefish and the Orca Model



The Orca hull in Stonefish (main hull selected: 36.6 t shell)

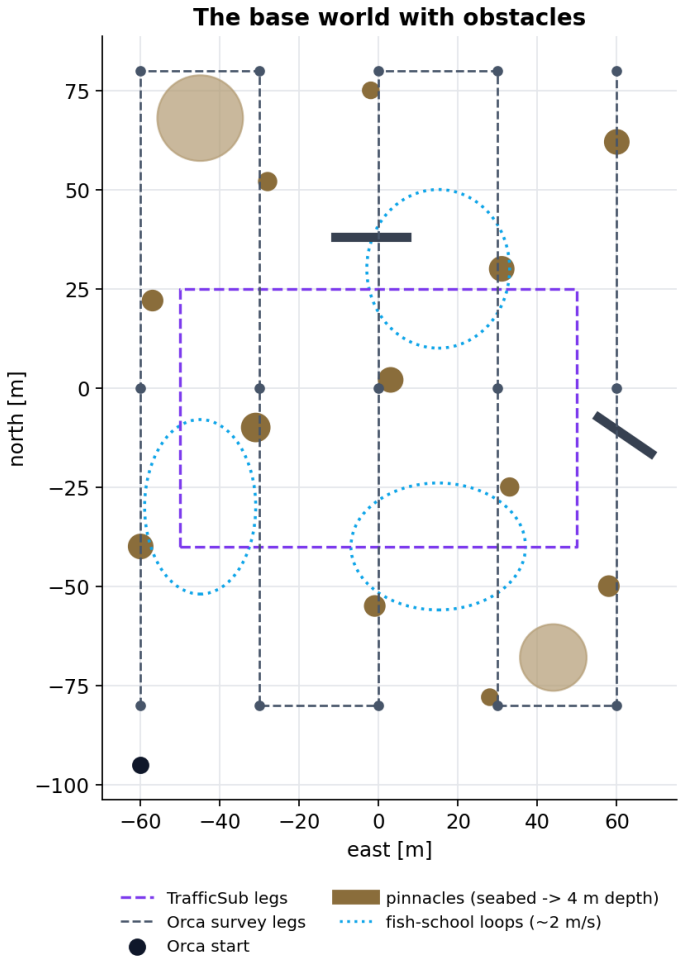
Thrusters

- Main thruster (±40 kN)
- Yaw thruster at the bow (±8 kN)
- Heave thruster through the centre of gravity (±55 kN)

Sensors

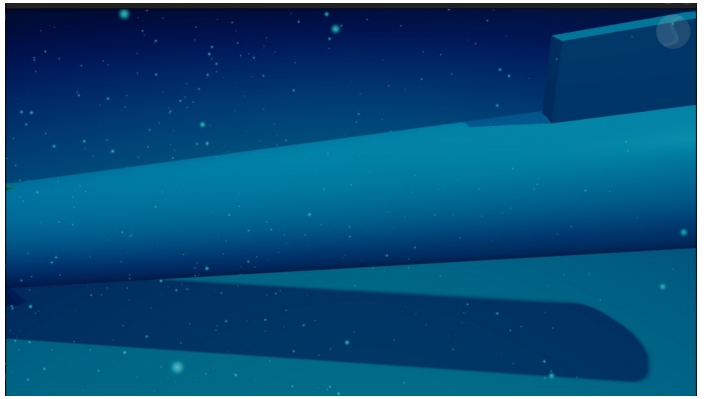
- Inertial unit · Doppler velocity log · pressure · GPS
- Profiling sonar: one swept beam, 160°, 30 m (obstacles)
- Multibeam sonar: 120° downward fan (seabed)

The Base World Now Has Obstacles



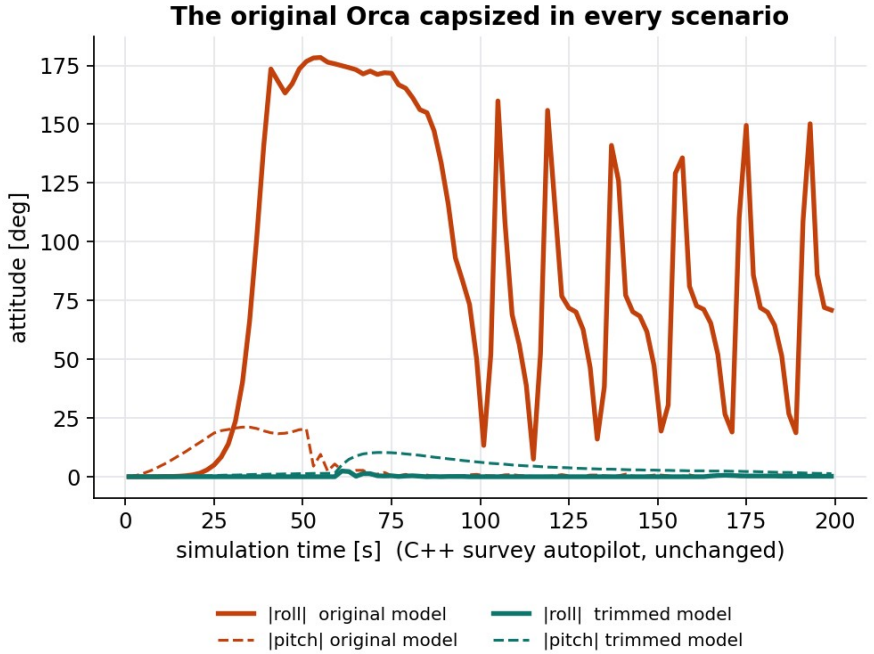
Obstacle world (generated procedurally)

- 12 rock pinnacles and 2 ridge walls rising from the 30 m seabed through the 12 m cruise depth
- 2 large boulders, 3 fish schools (9 fish, about 2 m/s loops)
- A traffic submarine crossing the lanes with its own avoidance



The Orca only learns about any of this through its own sonars and DVL.

Fix 1: The Original Orca Capsized



Mass properties read from Stonefish

	original	trimmed
mass [kg]	53 370	53 625
buoyancy [kg]	53 411	53 667
CG below CB [m]	-0.05	+0.08
CG - CB fore-aft [m]	+0.05	0.00
sail / mast	Composite	Syntactic foam
keel ballast	-	2 769 kg @ z=+1.05

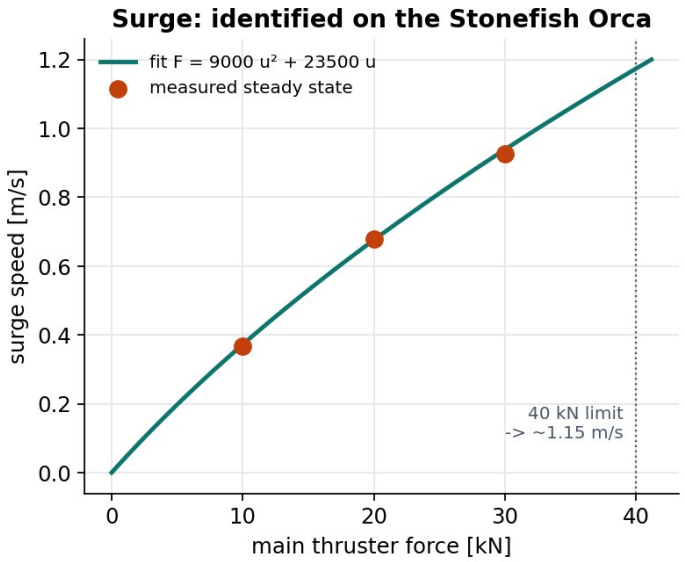
- Stonefish's own mass properties: the solid sail put the **centre of gravity 5 cm above the centre of buoyancy**.
- Every scenario, the original survey demo included, rolled past 170° within about 35 s.
- Fix: foam sail and mast; the removed mass carried as low **keel ballast**, placed so the two centres line up.
- Same net buoyancy; gravity now 8 cm below buoyancy. Same fix for the survey subs.

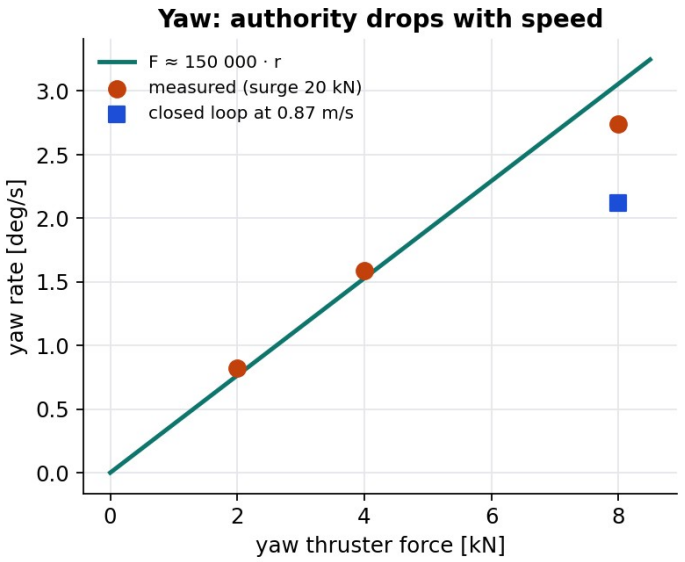
Verified: the unchanged survey autopilot now completes its lanes with roll ≈ 0° and pitch ≤ 10°.

Fixes 2-4 Found While Integrating

Problem	Root cause (from the Stonefish source)	Fix
DVL never had bottom lock	only a velocity range was given, leaving the altitude limit at about 10^{30}; the beam search then ran zero steps	explicit altitude range in the vehicle description
Seabed coverage stopped after one fan	the sensor interface returns sample copies whose identifier is always zero	detect new samples by their time stamp
Windowless mode aborted	lights are rejected when running without a window	remove lights after loading the scene
Orca flung out of the water	scripted fish squeeze the hull into the seabed or another hull; the physics engine diverges	watchdog: put the Orca back ~2 s earlier and report it
Two simulators on one network	interleaved sensor data, jumping estimate	warning in the run script; separate ROS domain per run

System Identification on the Stonefish Orca





- Raw thrust steps sent over ROS 2, with the vehicle reset between steps.
- The published Orca envelope (4.1 m/s, 3°/s) is **not achievable** by this model: 1.15 m/s at 40 kN.

1.15 m/s

top speed (40 kN)

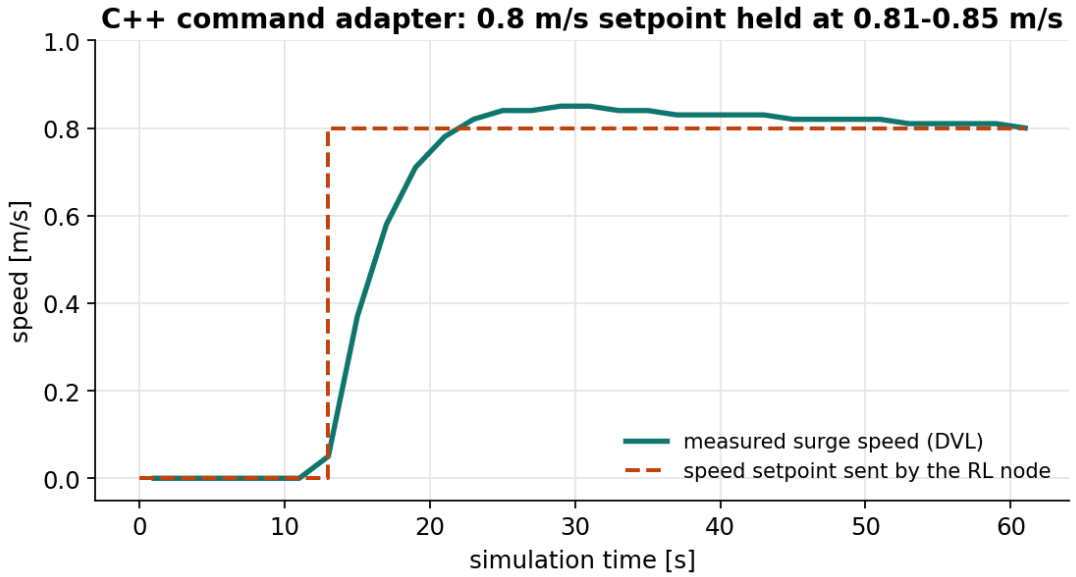
2.7 °/s

max yaw rate at 0.5 m/s

τ ≈ 6.5 s

surge time constant

From RL Action to Thruster Force

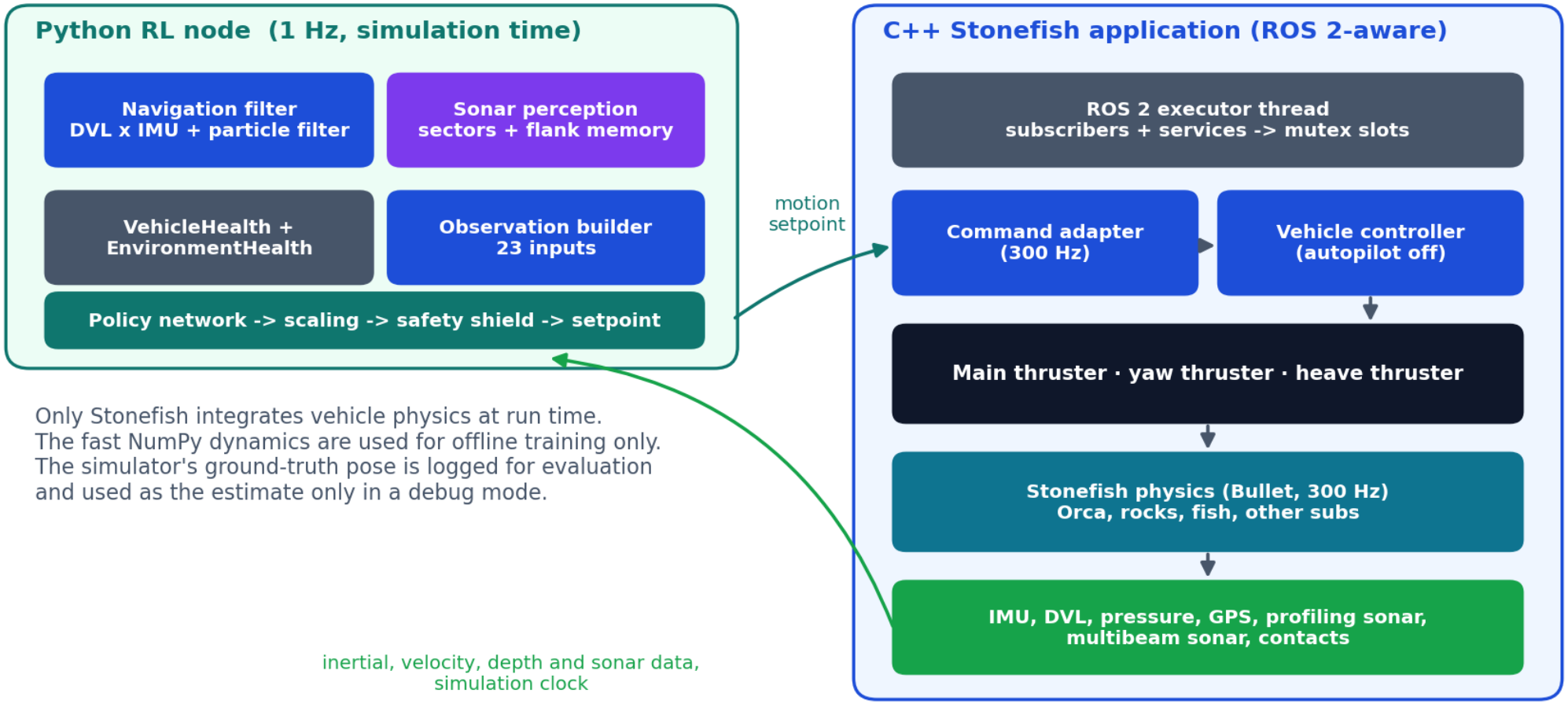


Actions are not forces: the policy asks for physical setpoints and the C++ inner loop turns them into thrust using the identified model.

Command adapter (C++, 300 Hz)

- Surge: $F = 9000 \cdot u|u| + 23500 \cdot u$ feed-forward + PI on DVL speed
- Yaw: $150\,000 \cdot r$ feed-forward + PI on the measured yaw rate
- Depth: integrates the depth-rate command into a reference, held with the autopilot's gains + seabed guard
- Drives the **existing** vehicle controller with its autopilot switched off
- 3 s watchdog: stop and hold depth if commands stop

Part C · How RL and Stonefish Connect



Only Stonefish integrates vehicle physics at run time.
The fast NumPy dynamics are used for offline training only.
The simulator's ground-truth pose is logged for evaluation
and used as the estimate only in a debug mode.

From Gazebo Assumptions to Stonefish

Old assumption (Gazebo / UUV Simulator)	Stonefish replacement
Ground-truth pose fed into the control loop	simulator odometry, labelled ground truth; used only for evaluation or in debug mode
Depth = (pressure - 1 atm) / ρg	Stonefish reports gauge pressure: depth = pressure / ρg (else 10 m error)
Velocity command to a simulator plugin	speed / yaw-rate / depth-rate setpoint, converted to thruster forces in C++
Per-thruster plugin messages	direct thruster forces, kept for manual tests
Fixed heading, zero velocity placeholders	measured heading and DVL velocity into the particle filter
Simulator reset / pause services	the robot is re-spawned at its start pose on the physics thread
Spawning the model into the world	the Orca is loaded from the scenario description
Procedural risk map	risk from the profiling sonar and DVL altitude; terrain richness from the multibeam

Data Exchanged over ROS 2

Data	Direction	Rate
Motion setpoint (speed, yaw rate, depth rate)	RL -> simulator	1 Hz
Attitude and turn rate (inertial unit)	simulator -> RL	100 Hz
Velocity over ground, altitude (DVL)	simulator -> RL	10 Hz
Depth (pressure)	simulator -> RL	10 Hz
Position fix (GPS, surface only)	simulator -> RL	1 Hz
Profiling and multibeam sonar scans	simulator -> RL	10 / 2 Hz
Contact events	simulator -> RL	10 Hz
Simulation clock	simulator -> RL	100 Hz
Reset, autopilot on/off	service	on request

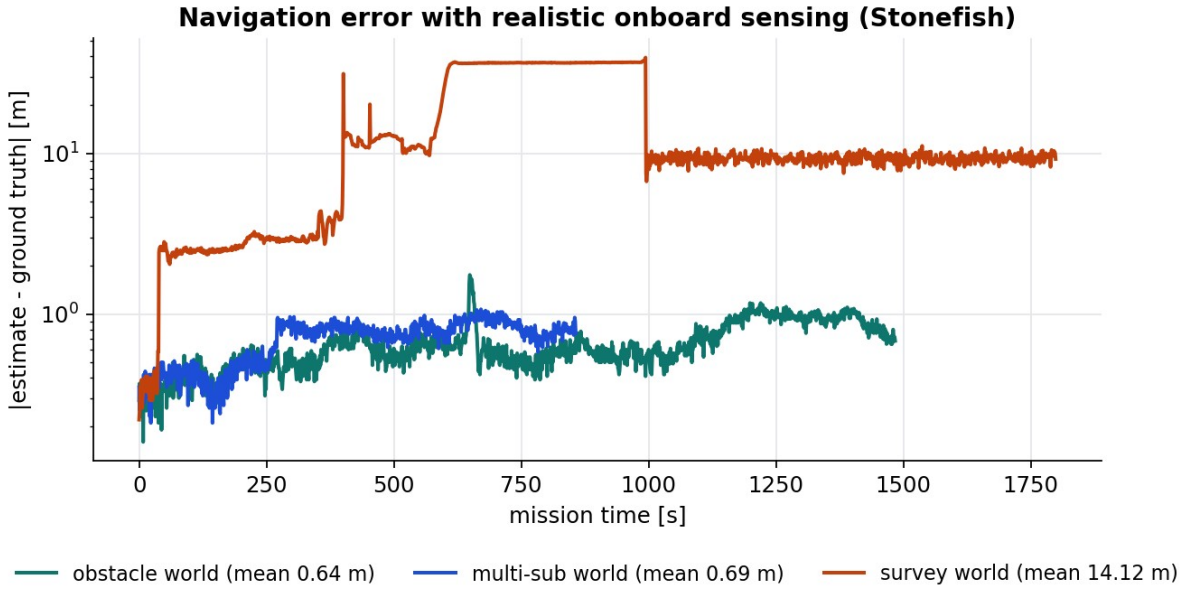
Frames (checked in the source)

- World north-east-down, body forward-right-down
- The Python simulator already uses the same axes
- + yaw = bow to starboard; depth positive down

Threads and timing

- Physics thread: 300 Hz step callback
- ROS 2 on its own thread
- Only lock-protected data is shared
- RL at 1 Hz on simulation time

Navigation: Realistic vs. Debug



Under 1 m mean error over 25-minute missions in the obstacle and multi-sub worlds. The survey-world error comes from physics resets, which navigation cannot observe.

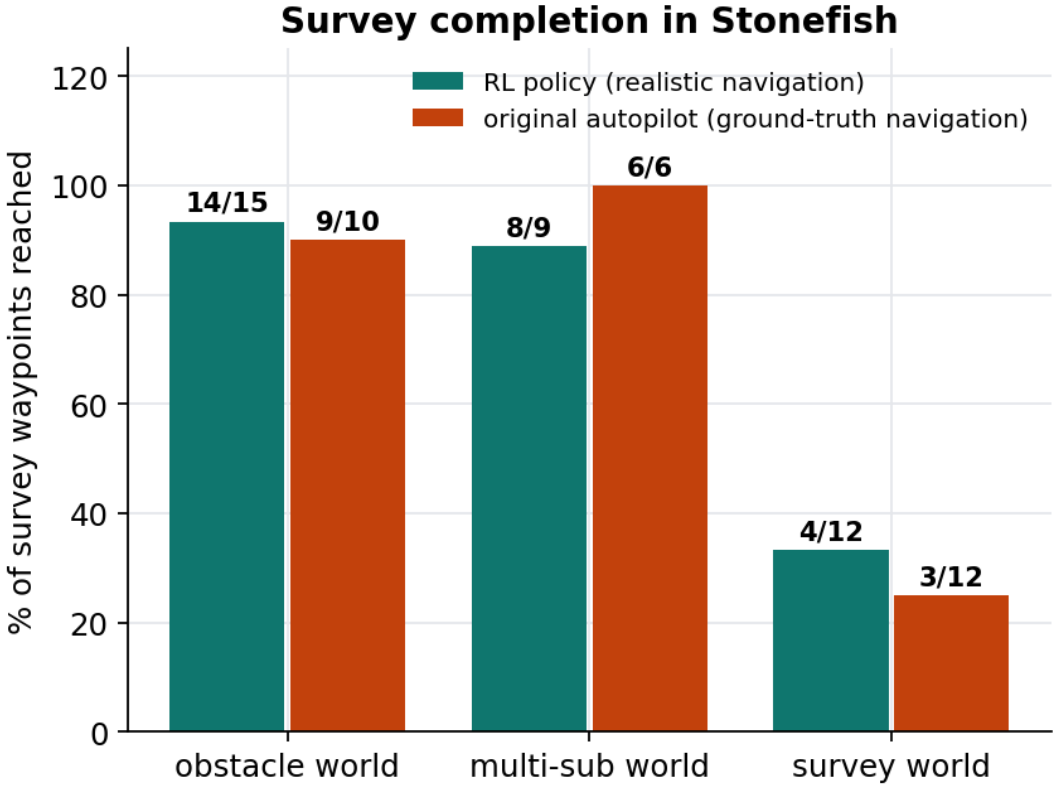
Realistic sensing (default)

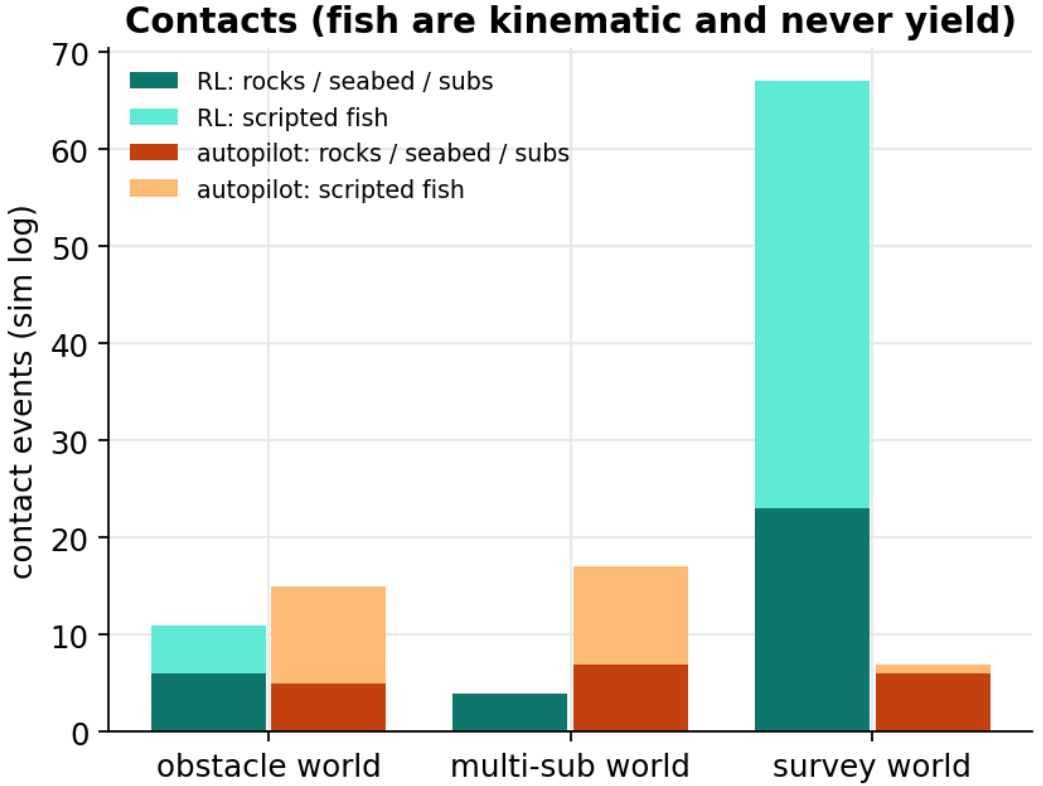
- DVL x inertial dead reckoning at 10 Hz
- Existing particle filter, GPS at the surface
- Ground truth logged only for evaluation

Debug: ground truth

- Simulator pose used as the estimate
- Connectivity checks only
- Warns in the log every 10 s

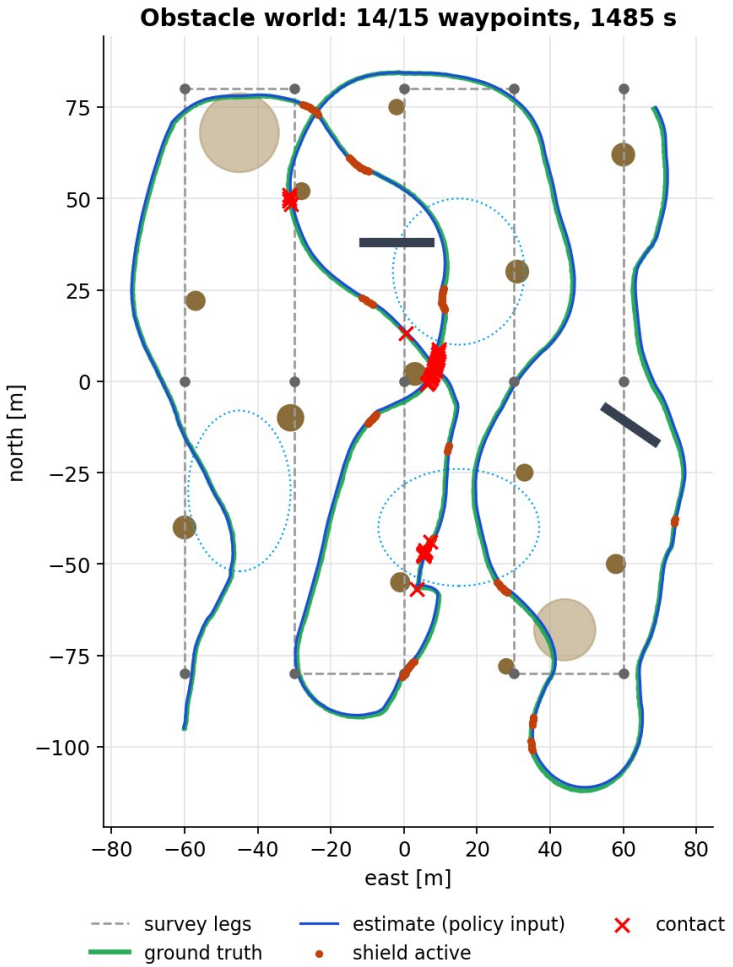
Closed-Loop Results in Stonefish





RL: realistic navigation, 30-minute limit. Autopilot: the original C++ controller using ground-truth position, in the same worlds.

Obstacle World: Full Survey Under RL Control



14 / 15

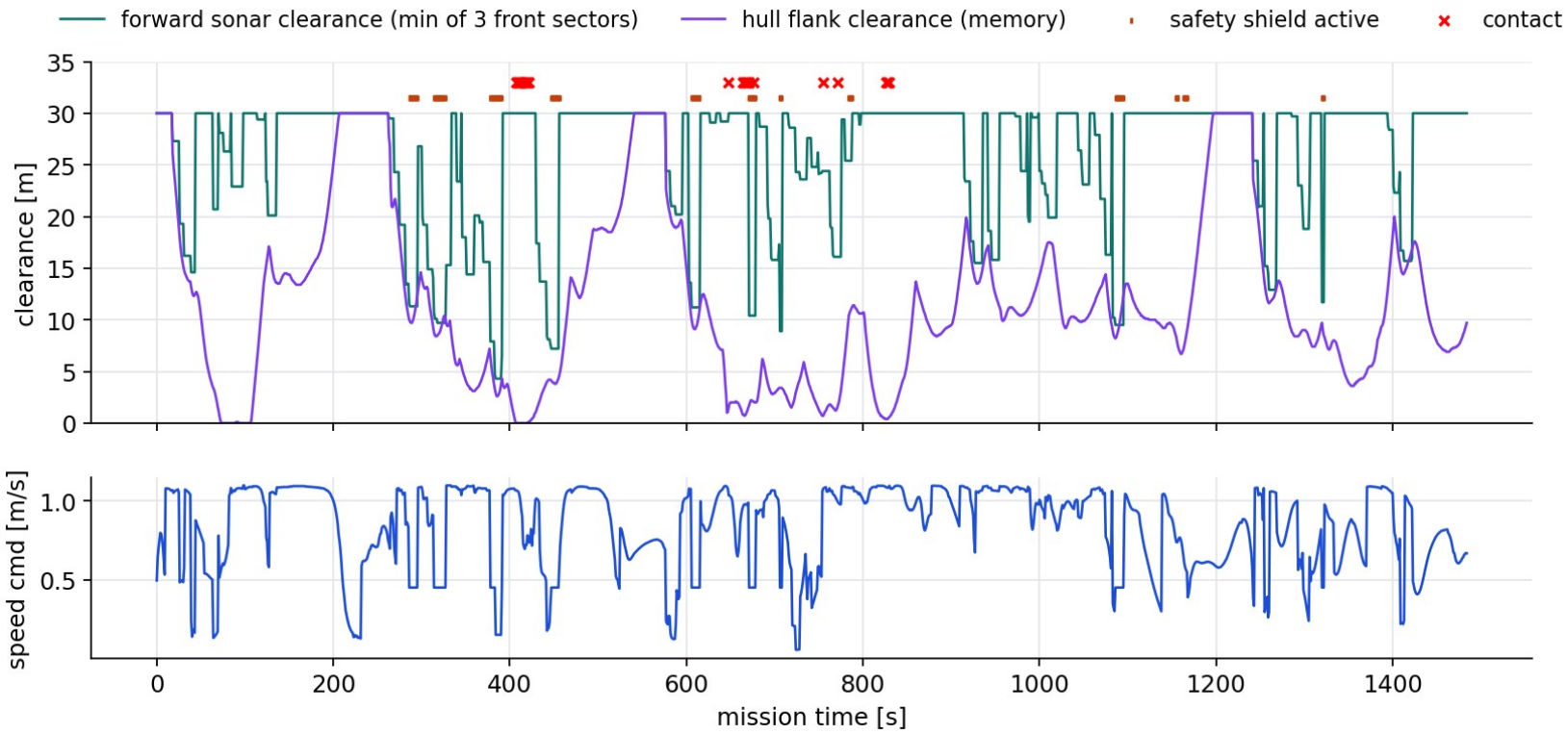
waypoints (1 skipped: on a rock)

0.64 m

mean navigation error

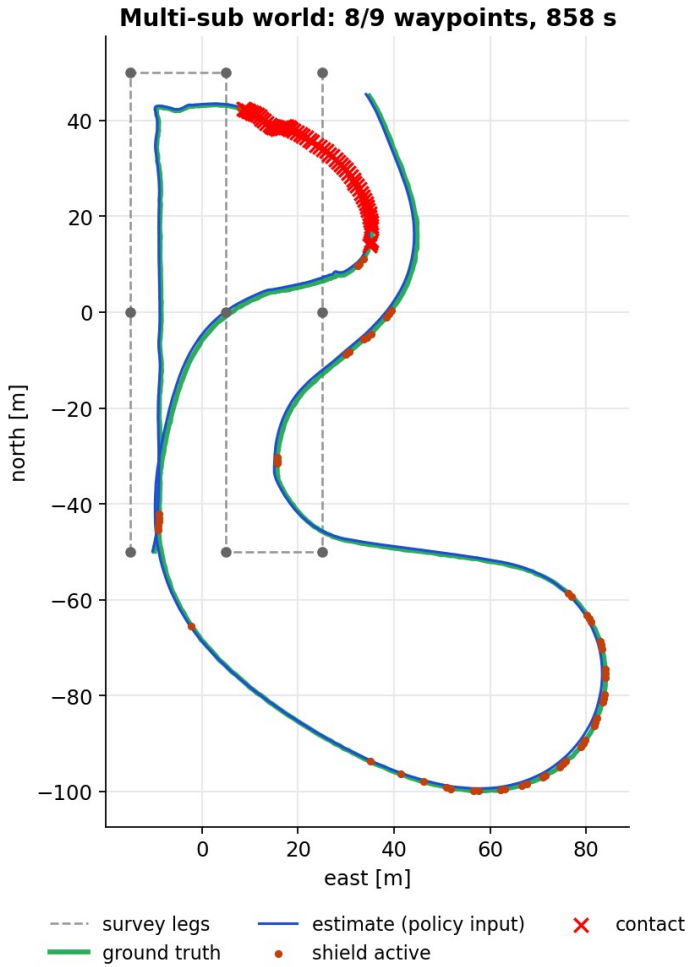
- Detours around pinnacles, ridges and boulders on every lane.
- 6 static contacts are grazes in tight passes; 5 are scripted fish crossing the hull.
- A waypoint the sonar shows to be occupied is skipped, as a real mission manager would.

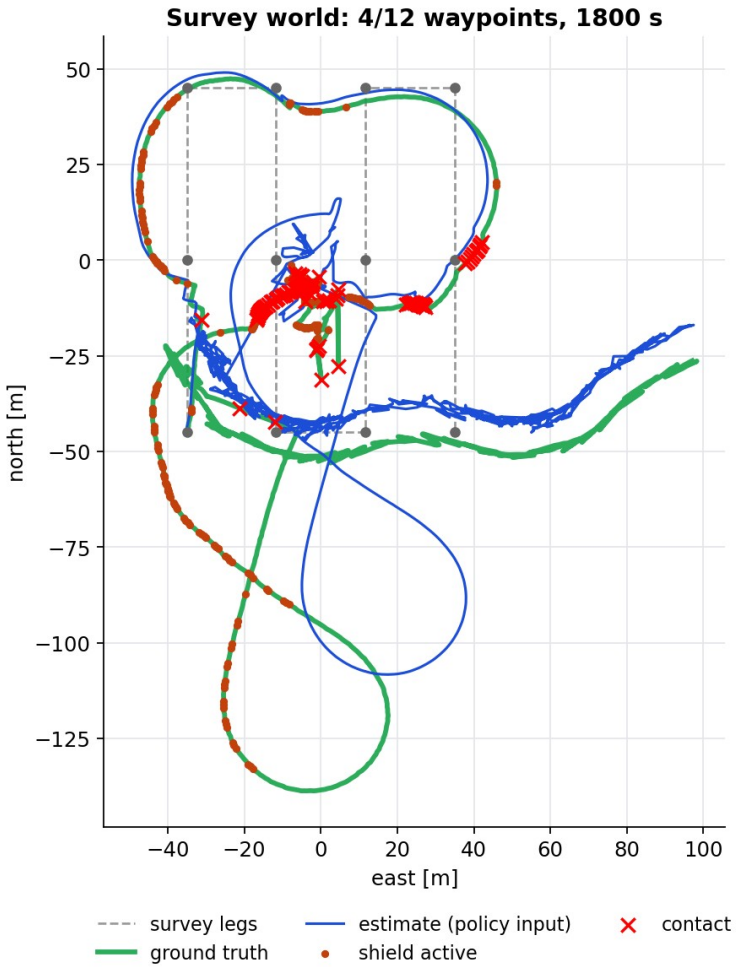
What the Policy Sensed and Did



Speed drops whenever forward clearance shrinks; the shield fires only in brief bursts; the flank memory keeps a rock alongside until the hull has passed it.

Other Submarines and the Dense Survey Field





Multi-sub world: complete

- 8/9 waypoints, 4 grazes with another sub, 0.69 m navigation error

Survey world: the open problem

- 90 x 70 m box for a 26 m hull, lanes 23 m apart
- Large scripted fish push the hull into the seabed: 15 physics resets, estimate drifts, legs time out
- The C++ autopilot also only reaches 3/12 there

Interim Summary / Conclusion

ROS 2	82 %	13 %	2 / 3
RL policy <-> Stonefish	offline goals (planner 68 %)	offline collisions (planner 28 %)	Stonefish surveys completed

- The policy drives the real Stonefish thrusters through ROS 2 and reads only real Stonefish sensors.
- The objective rewards progress and waypoints and penalises uncertainty, risk, energy, shield use and collisions.
- Three vehicle-model bugs were found and fixed: capsizing, DVL lock, seabed coverage.
- Remaining gap: dense scenes with non-yielding scripted fish, and grazing contacts in tight passes.

Limitations and Completion Plan

Limitations

- Trained in a fast simulator calibrated to Stonefish, not in Stonefish itself
- Flat seabeds: terrain-aided navigation carries no information
- Scripted fish never yield and can force physics resets
- The swept sonar refreshes each bearing only every ~6.4 s
- Contacts still occur: 6 grazes in the obstacle world

Next phase

- Fine-tune with ES directly in Stonefish, faster than real time, with automatic resets
- Randomise obstacle fields for every mission
- Real bathymetry charts so terrain-aided navigation helps
- Larger flank margin in tight passes
- PPO once a deep-learning toolchain is available

References

- T. Salimans et al., "Evolution Strategies as a Scalable Alternative to Reinforcement Learning," arXiv:1703.03864, 2017.
- J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- T. I. Fossen, Handbook of Marine Craft Hydrodynamics and Motion Control, Wiley, 2011.
- P. Cieślak, "Stonefish: An Advanced Open-Source Simulation Tool Designed for Marine Robotics, with a ROS Interface," IEEE/OES OCEANS, 2019.
- M. M. M. Manhães et al., "UUV Simulator: A Gazebo-Based Package for Underwater Intervention and Multi-Robot Simulation," IEEE/MTS OCEANS, 2016.
- S. Ross, G. Gordon, D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," AISTATS, 2011.
- S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.

Thank you

Questions?